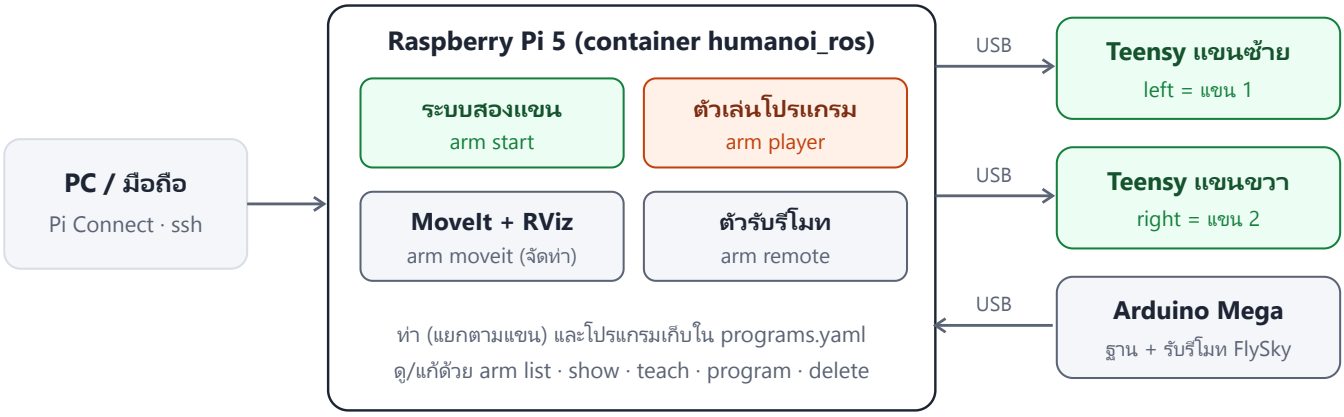
# คู่มือใช้งานแขนกล AR4 สองแขน

หุ่นยนต์ Humanoi · Raspberry Pi 5 · ROS 2 Jazzy · คำสั่งลัด **arm**

ฉบับ 19 ตุลาคม 2026 (สองแขน) · ไฟล์ต้นฉบับ docs/AR4_ARM_COMMANDS.md

## 0 ภาพรวมระบบ




หุ่นมีแขน AR4 สองตัวบนเสา เรียกตามด้านเมื่อ**มองจากตัวหุ่น**: **left** = แขน 1 (ซ้าย) พิมพ์ `left` `l` `1` หรือ `ซ้าย` · **right** = แขน 2 (ขวา) พิมพ์ `right` `r` `2` หรือ `ขวา`

ทุกคำสั่งในคู่มือนี้พิมพ์ใน **Terminal ของ Raspberry Pi นอก container** ด้วยคำสั่งลัด `arm` เปิด Terminal ได้ 2 ทาง:

- **Raspberry Pi Connect** → Screen sharing → กดไอคอน Terminal บนเดสก์ท็อป Pi (ต้องใช้ทางนี้ถ้าจะเปิด MoveIt)
- **จาก PC** → `ssh eitlaser@192.168.1.68`

ลืมคำสั่ง พิมพ์ `arm help` · ดูว่าอะไรเปิดอยู่ พิมพ์ `arm status`

### ① เปิด

1. `arm status`
2. `arm calibrate left` / `right` (เมื่อจำเป็น)
3. `arm start` หน้าต่าง 1
4. `arm player` หน้าต่าง 2

### ② ใช้

1. `arm run ชื่อโปรแกรม`
2. `arm moveit` จัดท่า
3. `arm teach left ชื่อท่า`
4. `arm remote 1` รีโมท

### ③ ปิด / หยุด

1. `arm stop` หยุดโปรแกรม
2. `arm goto left rest` / `right rest`
3. `arm off`
4. ปิดไฟแขน

> **ห้ามถอดหรือเสียบสาย USB ของแขนขณะไฟแขนเปิด** ปิดไฟแขนก่อนทุกครั้ง (Pi 5 ตัวแรกเสียหลังถอดสาย Teensy ตอนไฟแขนเปิดอยู่ เมื่อ 19 ต.ค. 2026)

> **หยุดฉุกเฉิน:** ปิดสวิตช์ไฟแขนเร็วที่สุด · คำสั่ง `arm stop` , ปุ่ม Stop ใน RViz และสวิตช์ SWD เป็นการหยุดทางซอฟต์แวร์ ใช้แทนสวิตช์ตัดไฟไม่ได้

## 1 ตั้งค่าครั้งแรก

ทำครั้งเดียวเมื่อ: ใช้ Pi ตัวใหม่ · เปลี่ยนบอร์ด Teensy · แฟลชเฟิร์มแวร์ใหม่ ชื่อแขนและทิศมอเตอร์เก็บอยู่ในบอร์ด Teensy ของแต่ละแขน (เฟิร์มแวร์ AR4_ROS2_seq v2) ย้ายสาย USB ไปช่องไหนของ Pi ก็หาเจอเอง

1. **เช็กไฟย้อนก่อนต่อแขนเข้า Pi ใหม่:** เปิดไฟแขน ยังไม่เสียบ USB เข้า Pi แล้ววัดแรงดันที่หัว USB-A ปลายสาย ระหว่างขา 5V (ขาริม) กับ GND (ขาริมอีกข้าง) ทุกเส้น (Teensy + Nano ทั้งสองแขน) **ต้องได้ 0V** ถ้าได้ ~5V ห้ามเสียบ ต้องแก้ก่อน (หรือใส่ USB isolator คั่น)
2. ปิดไฟแขน → เสียบ USB ทุกเส้นเข้า Pi → เปิดไฟแขน
3. `arm ports` ดูบอร์ด Teensy ทุกตัว: พอร์ต · เฟิร์มแวร์ ( `seq2` ) · ชื่อแขน · ข้อที่ตั้งกลับทิศ
4. `arm identify` ตั้งชื่อบอร์ด: ข้อ 6 (ข้อมือ) ของแต่ละบอร์ดหมุนไป-กลับ 10° ทีละบอร์ด ดูว่าแขนไหนขยับ แล้วตอบ `l` = ซ้าย · `r` = ขวา · `a` = ขยับอีกที · `s` = ข้าม จบแล้วขึ้น `left=... right=...` = ใช้ได้ (ไม่เห็นอะไรขยับ = ไฟแขนปิดอยู่)
5. **หาลำดับ calibrate ที่มือไม่ชน** (หัวข้อ 9) ไล่ทีละข้อ `arm calibrate left 6` , `arm calibrate left 5` , … ได้ลำดับที่ปลอดภัยแล้วตั้งเป็นค่าเริ่ม `LEFT_CAL_ORDER` / `RIGHT_CAL_ORDER` ใน `~/Humanoi/scripts/arm.sh`
6. ข้อไหนขึ้น `X ... FAIL` = มอเตอร์ข้อนั้นหมุนกลับด้าน → `arm dir right 000010` (ตัวอย่าง: J5 กลับทิศ) แล้ว calibrate ข้อนั้นใหม่
7. หลัง calibrate ครั้งแรก แขนอยู่ท่าพัก → `arm start` แล้วบันทึกท่าพักของแขนที่ยังไม่มี: `arm teach right rest`

> เช็กซ้ำว่าชื่อตรงกับแขนจริงได้ทุกเมื่อ: `arm identify left` ข้อ 6 ของแขนซ้ายจะหมุนให้ดูแขนเดียว (ระบบแขนต้องปิดอยู่)

## 2 เริ่มใช้งาน

คำสั่งที่ "เปิดค้าง" ใช้ **หน้าต่างละคำสั่ง** และห้ามปิดหน้าต่างระหว่างใช้งาน คำสั่ง `arm` จะไม่ยอมเปิดระบบแขนซ้อน 2 ชุด (เคยทำให้แขนขยับมั่ว)

| ขั้น | หน้าต่าง | คำสั่ง | หมายเหตุ |
|---|---|---|---|
| 0 | ไหนก็ได้ | `arm status` | ดูว่าอะไรเปิดอยู่และบอร์ดแขน ระบบแขนต้องเป็น *X* ก่อนเริ่ม |
| 1 | หน้าต่าง 1 | `arm calibrate left`<br>`arm calibrate right` | **เฉพาะเมื่อจำเป็น** (หัวข้อ 9) ทีละแขน จบเองเมื่อเสร็จ |
| 2 | หน้าต่าง 1 | `arm start` | เปิดระบบสองแขน **เปิดค้างไว้** · `arm start left` = แขนซ้ายจริง แขนขวาจำลอง |
| 3 | หน้าต่าง 2 | `arm player` | เปิดตัวเล่นโปรแกรม **เปิดค้างไว้** (ความเร็วเริ่มต้น 0.19) |
| 4 | หน้าต่าง 3 (เดสก์ท็อป Pi) | `arm moveit` | MoveIt + RViz สองแขน เฉพาะตอนจัดท่า (หัวข้อ 5) |
| 5 | หน้าต่าง 4 | `arm remote 1` | สั่งจากรีโมท เฉพาะตอนใช้รีโมท (หัวข้อ 7) |

| แบบต้อง calibrate | | |
|---|---|---|
| หน้าต่าง 1 | `arm calibrate left` | อ่านคำเตือน → Enter → รอจนขึ้น "calibrate ครบตามที่สั่งแล้ว" |
| หน้าต่าง 1 | `arm calibrate right` | เหมือนกัน |
| หน้าต่าง 1 | `arm start` | หน้าต่างเดิม เปิดค้างไว้ |
| หน้าต่าง 2 | `arm player` | เปิดค้างไว้ |
| หน้าต่าง 3 | `arm run ชื่อโปรแกรม` | ทดสอบ |

| แบบไม่ต้อง calibrate | | |
|---|---|---|
| หน้าต่าง 1 | `arm start` | เปิดค้างไว้ |
| หน้าต่าง 2 | `arm player` | เปิดค้างไว้ |
| หน้าต่าง 3 | `arm run ชื่อโปรแกรม` | ทดสอบ |

> `arm start` หาพอร์ตของแต่ละแขนเองจากชื่อในบอร์ด แล้วแสดงว่าใช้พอร์ตไหน ถ้าขึ้น `หาบอร์ด Teensy แขน... ไม่เจอ` ดูหัวข้อ 11

## 3 คำสั่งใช้งาน (หน้าต่างไหนก็ได้)

| ทำอะไร | คำสั่ง (ตัวอย่าง) |
|---|---|
| บันทึกท่าปัจจุบันของแขนหนึ่ง | `arm teach left pick_above` |
| สร้างโปรแกรมสองแขน (หัวข้อ 4) | `arm program pick2 rest/rest act1/act1 act2/-wait=1.0 rest/rest` |
| ดูโปรแกรมเป็นตาราง ชั้น \| ซ้าย \| ขวา | `arm show pick2` |
| ดูท่าทุกแขนและโปรแกรมทั้งหมด | `arm list` |
| เล่นโปรแกรม รอจนจบแล้วบอกผล ✓/✗ | `arm run pick2` |
| หยุดโปรแกรมกลางคัน (ทุกแขน) | `arm stop` |
| พาแขนเดียวไปท่าเดียว | `arm goto right pick_above` |
| ลบท่า (ต้องไม่มีโปรแกรมไหนใช้อยู่) | `arm delete left pick_above` |
| ลบโปรแกรม (ท่าในโปรแกรมยังอยู่) | `arm delete-program pick2` |
| ปิดทุกอย่างของแขน | `arm off` |

ท่าเก็บ**แยกตามแขน** ชื่อท่าซ้ำกันข้ามแขนได้ ( `rest` ของแขนซ้ายกับของแขนขวาเป็นคนละท่า) · ตั้งชื่อท่าและโปรแกรมเป็นภาษาอังกฤษหรือตัวเลข

## 4 เขียนโปรแกรมสองแขน

แต่ละขั้นเขียน **`ท่าแขนซ้าย/ท่าแขนขวา`**

| เขียน | ความหมาย |
|---|---|
| `act1/act2` | แขนซ้ายไป `act1` แขนขวาไป `act2` **เริ่มพร้อมกันและถึงพร้อมกัน** |
| `act1/-` | แขนซ้ายไป `act1` แขนขวา**อยู่นิ่ง** ( `-` หรือ `still` ) |
| `-/act3` | แขนซ้ายอยู่นิ่ง แขนขวาไป `act3` |
| `wait=1.0` | หยุดรอ 1 วินาที (ทั้งสองแขน) |

**กติกา**

- ขั้นถัดไปเริ่มเมื่อ**ทุกแขนถึงแล้ว**
- `-` ไม่ได้จำตำแหน่งไว้ แขนค้างอยู่ตรงที่ขั้นก่อนหน้าพาไป
- ขั้นแรกควรเป็นท่าที่รู้ตำแหน่งแน่นอนทั้งสองแขน เช่น `rest/rest`
- โปรแกรมแบบเก่า (ไม่มี `/` เช่น `demo` ) = แขนซ้ายอย่างเดียว ยังเล่นได้

**ตัวอย่าง `arm show pick2`**

```
โปรแกรม 'pick2' (5 ขั้น)
  ขั้น  แขนซ้าย (1)  แขนขวา (2)
     1  rest         rest
     2  act1         act1
     3  act2         still
     4  wait 1 s
     5  rest         rest
```

> **สำคัญ:** ไม่มีการเช็กการชนตอน `arm run` / `arm goto` (รวมสองแขนชนกันเอง) แขนเดินตรงจากท่าหนึ่งไปอีกท่า **ไม่ได้เดินตามเส้นทางที่ MoveIt วางไว้ตอนจัดท่า** ทดสอบโปรแกรมใหม่ด้วยความเร็วต่ำก่อน ถ้าเฉียดให้เพิ่มท่าคั่นกลาง เช่นยกแขนขึ้นก่อนหมุนฐาน

## 5 สร้างท่าด้วย MoveIt

### เปิด MoveIt

1. เปิดระบบแขนไว้ก่อน ( `arm start` ) MoveIt สั่งแขนผ่านระบบนี้
2. เปิด **Raspberry Pi Connect** → **Screen sharing** ให้เห็นเดสก์ท็อป Pi
3. บนเดสก์ท็อป Pi กดไอคอน **Terminal** (รูปจอดำ `>_` แถบบนซ้าย)
4. พิมพ์ `arm moveit` แล้ว Enter รอ 20–30 วินาที หน้าต่าง **RViz** ขึ้นพร้อมโมเดลสองแขน เสา หัว และกล้อง
5. ปล่อยหน้าต่าง Terminal นี้เปิดไว้ (ปิด MoveIt = `Ctrl+C` ในหน้าต่างนี้)

ขึ้น `✗ เปิดระบบแขนก่อน` = ยังไม่ได้ `arm start` · RViz ไม่ขึ้น = เดสก์ท็อป Pi ยังไม่ได้ล็อกอิน (ต้องเห็นหน้าจอใน Pi Connect ก่อน)

### จัดท่าและบันทึก

1. แผง MotionPlanning → ช่อง **Planning Group** เลือก `left_arm` , `right_arm` หรือ `both_arms`
2. แท็บ **Joints** เลื่อนมุมทีละแกน (หรือลากลูกบอลที่ปลายแขน) แขนสีส้มโปร่งขยับให้ดูก่อน **แขนจริงยังไม่ขยับ**
3. แท็บ **Planning** → Velocity Scaling ตั้งไว้ **0.3** (อย่าเกิน) → กด **Plan & Execute** → **แขนจริงขยับ** · ตอน Plan, MoveIt เช็กการชนแขนกับแขน และแขนกับเสา/หัว/กล้องให้ ถ้าชนจะ Plan ไม่ผ่าน
4. แขนถึงท่าแล้ว ในอีกหน้าต่างพิมพ์ `arm teach left ชื่อท่า` (หรือ `right` )
5. ทำซ้ำทุกท่า → `arm program ...` → ทดสอบ `arm run ชื่อโปรแกรม`

| ลำดับหน้าต่างตอนจัดท่า | | |
|---|---|---|
| หน้าต่าง 1 | `arm start` | เปิดค้าง |
| หน้าต่าง 2 | `arm player` | เปิดค้าง |
| หน้าต่าง 3 (เดสก์ท็อป Pi) | `arm moveit` | เปิดค้าง RViz ขึ้นบนจอ Pi |
| หน้าต่าง 4 | `arm teach / program / run` | สั่งงาน |

ปุ่ม **Stop** ในแผง Planning หยุดแขนกลางทางได้ · ฉุกเฉินปิดสวิตช์ไฟแขน

## 6 แก้ไขและลบ ท่า/โปรแกรม

| อยากทำ | ทำยังไง |
|---|---|
| **แก้ท่าเดิม** | `arm goto แขน ท่า` → ปรับใน MoveIt (Joints → Plan & Execute) → `arm teach แขน ชื่อเดิม` บันทึกทับ ระบบแสดงค่าเก่า/ใหม่ให้เทียบ ทุกโปรแกรมที่ใช้ท่านี้ได้ค่าใหม่ทันที |
| **แก้ลำดับในโปรแกรม** | `arm program ชื่อเดิม ชั้นใหม่ทั้งหมด...` สั่งทับได้เลย |
| **ลบท่าที่โปรแกรมยังใช้** | ระบบไม่ยอมและบอกว่าโปรแกรมไหนใช้อยู่ แก้หรือลบโปรแกรมนั้นก่อน แล้วค่อย `arm delete แขน ท่า` |
| **เปลี่ยนชื่อท่า** | `arm goto แขน ชื่อเก่า` → `arm teach แขน ชื่อใหม่` → แก้โปรแกรมให้ใช้ชื่อใหม่ → `arm delete แขน ชื่อเก่า` |
| **ลบโปรแกรม** | `arm delete-program ชื่อ` (ท่าที่อยู่ในโปรแกรมไม่หาย) |

> แก้ท่าหรือโปรแกรมแล้วไม่ต้องรีสตาร์ทอะไร ตัวเล่นโปรแกรมอ่านไฟล์ใหม่ทุกครั้งที่สั่ง

## 7 สั่งจากรีโมท FlySky

| สวิตช์ | ทำอะไร |
|---|---|
| **CH6** = **SWC** (3 ระดับ)<br>บนสุด → ล่างสุด | เล่นโปรแกรม `1` หนึ่งครั้ง (เปลี่ยนได้ เช่น `arm remote pick2`) ตำแหน่งกลางไม่นับ ค้างล่างสุดไม่เล่นซ้ำ จะเล่นใหม่ต้องดันขึ้นบนสุดก่อน |
| **CH8** = **SWD**<br>ขึ้น → ลง | หยุดแขนทันที (ทุกแขน) และระหว่างที่ SWD ลงค้างอยู่ สั่งเล่นไม่ได้ |
| **SWA** | ARM ล้อฐาน (ไม่เกี่ยวกับแขน) ระหว่างใช้แขน ให้ SWA ขึ้นไว้ ล้อจะล็อกนิ่ง |

**ต้องมี:** Mega เสียบ USB เข้า Pi · รีโมทเปิดอยู่ · `arm start` และ `arm player` เปิดอยู่ แล้วเปิดอีกหน้าต่าง:

| | | |
|---|---|---|
| หน้าต่าง 4 | `arm remote 1` | เปิดค้างไว้ แสดงทุกครั้งที่โยกสวิตช์ และผลจากแขน ( `แขนรายงาน: 1:ok` ) |

### ระบบกันสั่งมั่ว

- สั่งเฉพาะ**จังหวะโยก** สวิตช์ค้างไว้ไม่สั่งซ้ำ
- เปิดระบบตอนสวิตช์อยู่ล่าง = ไม่สั่ง ต้องเห็นสวิตช์อยู่บนก่อน
- รีโมทดับหรือหลุดสัญญาณ = ไม่สั่ง และหลังสัญญาณกลับมาต้องดันสวิตช์ขึ้นก่อน

- แขนกำลังเล่นโปรแกรมอยู่แล้วสั่งซ้ำ = ถูกปฏิเสธ ( `rejected:busy` ) ไม่ซ้อนกัน

`arm remote` เปิด `relay_bridge` (คุยกับ Mega) ให้ด้วยถ้ายังไม่ได้เปิด กด Ctrl+C ปิดทั้งคู่ · รีโมทยังขับฐานได้ตามปกติ · ถ้ายังไม่ได้เปิด `arm player` ก็ทดสอบสวิตช์ได้ เห็นผลบนจอแต่แขนไม่ขยับ

## 8 ปิดระบบ

1. ให้แขนกลับท่าพักก่อน: `arm goto left rest` และ `arm goto right rest` (หรือสร้างโปรแกรม `arm program park rest/rest` แล้ว `arm run park` )
2. `arm off` ปิดทุกอย่างของแขน (หรือกด Ctrl+C ทีละหน้าต่าง)
3. ปิดไฟแขน

## 9 Calibrate

**ต้อง calibrate เมื่อ**

- ปิดไฟแขน (รวมทุกครั้งที่ต้องถอด/เสียบ USB เพราะต้องปิดไฟแขนก่อน)
- แฟลชเฟิร์มแวร์ Teensy ใหม่ หรือเปลี่ยนทิศมอเตอร์ด้วย `arm dir`
- แขนชนอะไร หรือสายพานรูด

Teensy นับตำแหน่งจากสเต็ปที่สั่งมอเตอร์ ไม่ได้วัดจริง ถ้าเกิดเรื่องข้างบน ตำแหน่งที่จำไว้จะไม่ตรงกับแขนจริง ถ้าไม่เกิดให้ข้ามไป `arm start` ได้เลย · calibrate ได้เฉพาะตอนระบบแขนปิดอยู่

**ระหว่าง calibrate**

- ครั้งแรกหลังย้ายสาย ดูให้แน่ใจว่าแขนที่ขยับคือแขนที่สั่ง ไม่ใช่ หรือมือกำลังจะชน ให้ปิดไฟแขนทันที
- รอบแขนโล่ง สาย USB หย่อนพันรัศมีแขน ฐาน J1 หมุนไปถึง −170° อีกแขนต้องไม่อยู่ในทาง
- ข้อที่ไม่ได้สั่งไม่ขยับ แต่ถ้าเพิ่งเปิดไฟแขน ต้อง calibrate ให้ครบ 6 ข้อก่อนใช้งาน
- จบเมื่อขึ้น `calibrate ครบตามที่สั่งแล้ว` **ไม่ต้องกด Ctrl+C**

### สั่งยังไง (ทีละแขน)

| คำสั่ง | ทำอะไร |
|---|---|
| `arm calibrate left` | ใช้ลำดับที่ตั้งไว้ใน `LEFT_CAL_ORDER` (ยังไม่ตั้ง = ต้องใส่ลำดับเอง) |
| `arm calibrate left 5` | ข้อเดียว ไว้ไล่ดูว่าข้อไหนพามือไปชน |
| `arm calibrate left 6 5 4 1 2 3` | ทีละข้อตามลำดับ แต่ละข้อหาสวิตช์ → ถอย → กลับท่าพัก ก่อนไปข้อถัดไป |
| `arm calibrate left 456 123` | กลุ่มละหลายข้อพร้อมกัน |
| `arm calibrate left all` | 6 ข้อพร้อมกันแบบต้นฉบับ (มือที่ติดปลายแขนอาจชน) |

### อ่านผลระหว่าง calibrate

| ขึ้นว่า | แปลว่า / ทำอะไร |
|---|---|
| `✓ J5 เจอสวิตช์แล้ว` | ข้อนั้นผ่าน |
| `✗ J5 ... FAIL` | หมุนครบระยะแล้วไม่เจอสวิตช์ บอร์ดหยุดข้อนั้นให้เอง (ไม่หมุนต่อจนชน) โปรแกรมจบพร้อมแจ้ง มักเป็นมอเตอร์ข้อนั้นหมุนกลับด้าน → `arm dir` แล้ว calibrate ข้อนั้นใหม่ · หรือสวิตช์/สายหลุด |
| `USB หลุดระหว่าง calibrate` | ปกติตอนมอเตอร์หมุน แขนทำต่อเอง โปรแกรมรอต่อใหม่ให้ |

**ทิศมอเตอร์ `arm dir`**

`arm dir left|right J1..J6` 6 หลัก 0/1 ตามลำดับ J1..J6, `1` = ข้อนั้นมอเตอร์หมุนกลับด้านจากปกติ เช่น `arm dir right 000010` = J5 แขนขวากลับทิศ · `arm dir right 000000` = ปกติทุกข้อ เก็บในบอร์ดของแขนนั้น ใช้ทั้งตอน calibrate และตอนวิ่ง ตั้งแล้วต้อง calibrate ข้อนั้นใหม่

## 10 ค่าที่ตั้งไว้และความปลอดภัย

| รายการ | ค่า / หมายเหตุ |
|---|---|
| ความเร็วตัวเล่นโปรแกรม | **0.19 rad/s** (ฐานหมุน ~11°/วินาที) เพดาน 0.22 สั่งเกินจะถูกลดลงเอง เร่งและเบรก 4.6°/วินาที² · สองแขนในขั้นเดียวกันถูกยืดเวลาให้ถึงพร้อมกัน |
| ความเร็ว MoveIt | Velocity Scaling **0.3** (อย่าตั้งเกิน) |
| การเช็กการชน | มีเฉพาะตอน Plan ใน MoveIt (แขนกับแขน, แขนกับเสา/หัว/กล้อง) · `arm run` / `arm goto` ไม่เช็ก |
| สายพาน J1 | เคยรูดตอนแขนยื่นไกลแล้วหมุนฐานเร็ว ท่าที่ยื่นแขนสุดและหมุนฐานไกล (เช่น `home`, `pick_frot`) ให้ทดสอบช้าๆ และให้ช่างตรวจความตึงสม่ำเสมอ |
| ท่าพัก ( `rest` ) | J2 = −5.8°, J5 = −52.2° แกนอื่น 0° (แขนอยู่ท่านี้หลัง calibrate) |
| สาย USB ของแขน | ถอด/เสียบเฉพาะตอนไฟแขนปิด · ต่อเข้า Pi ใหม่ต้องวัดไฟย้อนก่อน (หัวข้อ 1) |
| การหยุดฉุกเฉินจริง | สวิตช์ตัดไฟแขน · คำสั่งหยุดทางซอฟต์แวร์ใช้แทนไม่ได้ |

**ขอบเขตแต่ละแกน (ตัวเล่นโปรแกรมตรวจทุกท่าก่อนขยับ — สองแขนเท่ากัน)**

| แกน | J1 ฐาน | J2 ไหล่ | J3 ศอก | J4 | J5 | J6 |
|---|---|---|---|---|---|---|
| องศา | ±170 | −42 ถึง +90 | −89 ถึง +52 | ±165 | ±105 | ±155 |

## 11 แก้ปัญหา

| อาการ | สาเหตุ | แก้ |
|---|---|---|
| `Waiting for at least 1 matching subscription(s)` | ตัวเล่นโปรแกรมไม่ได้เปิด | `arm player` ในอีกหน้าต่าง |
| **`ระบบแขนเปิดอยู่แล้ว — ห้ามเปิดซ้อน`** | มีระบบแขนเปิดอยู่แล้ว | ใช้ตัวเดิม หรือ `arm off` แล้วเริ่มใหม่ |
| `Controller already loaded`, `Unknown header C1/P0/JA` | เปิดระบบแขนซ้อน 2 ชุด | `arm off` → calibrate ใหม่ |
| **`หาบอร์ด Teensy แขน... ไม่เจอ`** | บอร์ดยังไม่ได้ตั้งชื่อ / ไฟแขนปิด / ค้างอยู่ใน calibrate | `arm ports` ดูก่อน · ยังไม่มีชื่อ → `arm identify` · ค้าง → ปิด-เปิดไฟแขน |
| `arm ports` ขึ้น **`ไม่ตอบ`** | บอร์ดกำลัง calibrate หรือระบบแขนเปิดใช้พอร์ตอยู่ | รอ calibrate จบ / `arm off` |
| **`มีบอร์ดชื่อ 'left' สองตัว`** | ตั้งชื่อซ้ำ | `arm identify` ใหม่ |
| calibrate ขึ้น `X J... FAIL` | มอเตอร์ข้อนั้นหมุนกลับด้าน หรือสวิตช์/สายหลุด | `arm dir แขน 6หลัก` → calibrate ข้อนั้นใหม่ |
| calibrate แล้วมือชน | หลายข้อหมุนพร้อมกัน | calibrate ทีละข้อ หาลำดับที่ไม่ชน แล้วตั้ง `LEFT_CAL_ORDER` / `RIGHT_CAL_ORDER` |
| แขนจริงหมุนกลับทิศกับโมเดลใน RViz | ทิศข้อนั้นในบอร์ดกับในโมเดลไม่ตรงกัน | ตรวจข้อนั้นด้วย calibrate ก่อน แล้วแจ้งให้แก้ (อย่าแก้ไฟล์โมเดลเอง) |
| `path tolerance violation` | สั่งเร็วเกิน มอเตอร์ตามไม่ทัน | ใช้ความเร็วปกติ (0.19) ห้ามเกิน 0.22 |
| สายพานรูด / ท่าเพี้ยน | แขนยื่นไกลแล้วเร่ง-หยุดแรง หรือสายพานหย่อน | ให้ช่างตั้งความตึง → calibrate ใหม่ |
| **`แขนขวาไม่มีท่า '...'`** | ท่านั้นยังไม่ได้ teach ให้แขนนั้น (ท่าแยกตามแขน) | `arm list` ดูชื่อท่าของแต่ละแขน → `arm teach แขน ท่า` |
| โยกสวิตช์รีโมทแล้วแขนไม่ขยับ | ยังไม่ได้ดันสวิตช์ขึ้นก่อน / SWD ลงอยู่ / ตัวเล่นโปรแกรมไม่เปิด | ดูหน้าต่าง `arm remote` · ดัน SWC และ SWD ขึ้น แล้วโยก SWC ลงใหม่ |
| `arm remote` ขึ้น **`ไม่เจอ Mega`** | Mega ไม่ได้เสียบที่ Pi | ปิดไฟแขน → เสียบ USB ของ Mega เข้า Pi |
| RViz ไม่ขึ้น | เดสก์ท็อป Pi ยังไม่ได้ล็อกอิน | เปิด Pi Connect ให้เห็นหน้าจอก่อน แล้ว `arm moveit` ใหม่ |

## 12 ภาคผนวก

### ซิมสองแขนบน PC (ไม่ต้องมี Pi / แขนจริง)

1. ดับเบิลคลิก `C:\Users\User\Desktop\ar4\run_dual_sim.bat` เปิด 3 หน้าต่าง: แขนจำลอง · MoveIt + RViz · ตัวเล่นโปรแกรม
2. RViz เปิดมาเป็นหน้าเปล่า (ตั้งใจ กันเมาส์ค้างใน WSL) → **File > Open Config** → `/mnt/c/Users/User/Desktop/ar4/ar4_ws/install/ar_dual/share/ar_dual/config/dual_moveit.rviz`

3. สอนท่า/สร้างโปรแกรมด้วย `ros2 run ar_program teach ...` ในหน้าต่าง WSL อีกหน้าต่าง (ตัวอย่างขึ้นในหน้าต่าง .bat) · ปิด = ปิดทั้งสามหน้าต่าง

### คำสั่งเต็ม (ใช้เมื่อคำสั่ง `arm` ใช้ไม่ได้)

เข้า container ก่อนทุกหน้าต่าง: `docker exec -it humanoi_ros bash`

```
# หาพอร์ต / ตั้งชื่อบอร์ด (ระบบแขนต้องปิดอยู่)
ros2 run ar_dual ar_ports.py --list
ros2 run ar_dual ar_ports.py                # พิมพ์ left=<พอร์ต> right=<พอร์ต>
ros2 run ar_dual ar_ports.py --identify
ros2 run ar_dual ar_ports.py --dir <พอร์ต> 000010
# calibrate (ทีละแขน)
ros2 run ar_dual ar_calibrate.py <พอร์ตแขน> 6 5 4 1 2 3
# เปิดระบบสองแขน (left_hw/right_hw = real | mock)
ros2 launch ar_dual dual_hardware.launch.py left_port:=<พอร์ตซ้าย> right_port:=<พอร์ตขวา> left_hw:=real righ
t_hw:=real
# ตัวเล่นโปรแกรม
ros2 launch ar_program arm_program.launch.py speed:=0.19
# สั่งงาน
ros2 topic pub --once /arm/run std_msgs/msg/String "{data: pick2}"
ros2 topic pub --once /arm/run std_msgs/msg/String "{data: '@right:rest'}"   # = arm goto right rest
ros2 topic pub --once /arm/stop std_msgs/msg/Empty "{}"
ros2 run ar_program teach --list
ros2 run ar_program teach --show pick2
ros2 run ar_program teach <ชื่อท่า> --arm left
ros2 run ar_program teach --program pick2 rest/rest act1/- wait=1.0 -/act2
ros2 run ar_program teach --delete <ชื่อท่า> --arm right
ros2 run ar_program teach --delete-program pick2
```

### ตำแหน่งไฟล์บน Pi

| อะไร | ที่อยู่ |
|---|---|
| ท่าและโปรแกรม | `~/Humanoi/ros2_ws/src/ar_program/config/programs.yaml` |
| คำสั่งลัด `arm` | `~/Humanoi/scripts/arm.sh` (alias ใน `~/.bashrc`) · ลำดับ calibrate ตั้งที่ `LEFT_CAL_ORDER` / `RIGHT_CAL_ORDER` |
| เปิด MoveIt | `~/Humanoi/scripts/arm_moveit.sh` |
| คู่มือ | `~/Humanoi/docs/AR4_ARM_COMMANDS.md` และไฟล์ PDF นี้ |
| Teensy สองแขน | หาเองจากชื่อที่เก็บในบอร์ด (`arm ports`) · บังคับเองได้: `LEFT_TEENSY=... RIGHT_TEENSY=... arm start` |
| Arduino Mega (ฐาน) | `/dev/serial/by-id/usb-Arduino__www.arduino.cc__0042_85830303393517150C0-if00` |

การรัน `scripts/sync_to_pi.sh` จาก PC จะเขียนทับ programs.yaml บน Pi (สำรองไว้เป็น programs.yaml.bak) ถ้า teach ท่าไว้บน Pi ให้ดึงไฟล์กลับมา PC ก่อน · ไฟล์ท่าของซิมบน PC (`ar4\ar4_ws\src\ar_program\config\programs.yaml`) เป็นคนละไฟล์กับบน Pi